

# A Cryptodiagnosis of Cicada 3301’s Unsolved *Liber Primus*

Jens Wedin

Independent researcher

<https://github.com/jens-wedin/liber-primus>

Draft — August 24, 2026

## Abstract

The *Liber Primus* is the runic codex at the centre of the Cicada 3301 puzzles. Its second release (LP2, 2014) is 58 pages; only two are solved. The unsolved remainder is a stream of 12,956 runes over a 29-letter alphabet. We give a control-validated diagnosis of that stream.

The ciphertext is statistically a uniform random stream with one constraint: adjacent runes are almost never equal. The doublet rate is 0.66% against a 3.45% random expectation — a deficiency of about 17 standard deviations. First differences are otherwise uniform. This is a pure lag-1 effect. It is the only structure in the stream, and it is reproduced independently by three parties.

We show that this single anomaly is the signature of an output-stage no-repeat rule, most plausibly a *soft* rejection that keeps a would-be doublet with probability  $p \approx 0.19$ . Such a rule desynchronises any keystream by about 3% and defeats every fixed-position attack. We then eliminate, each against a planted positive control and a matched chance ceiling, every classical and modern cipher class we could formulate: substitution and shift families; periodic and running keys, including the texts Cicada is documented to have used; autokey; affine and other two-variable functions; difference-space cumulative ciphers; seeded pseudo-random and hash-derived pads; hand-computable (Gromark) keystreams; magic squares and code pages as keys; a generalised interrupter over all 29 runes; and transposition and orientation variants. Two independent solver projects reach the same negative.

We give the information-theoretic footing. At 12,956 runes the plaintext is far past unicity for any short-seed scheme, so unbreakability can only be computational. The remaining wall is a high-entropy or external keyed pad; its seed entropy, not the pad idea, is what resists. We argue that the way around the wall is the puzzle’s non-runic numeric content, which sits behind its own pad. The contribution is a narrowed hypothesis space and a reusable methodology, not a break.

## 1 Introduction

Cicada 3301 posted three sets of cryptographic puzzles, on or near 4 January of 2012, 2013 and 2014, plus later signed messages. The stated aim was to recruit “intelligent individuals.” The group’s identity is unverified. The 2014 round led to the *Liber Primus*, a codex written in a bespoke 29-rune alphabet called Gematria Primus. The community splits it into LP1 (17 pages, all solved) and LP2 (58 pages). Only two LP2 pages were ever solved. The rest have resisted a decade of collective effort, including a 2023 DEF CON status talk that reported no progress.

We do not claim a solution. We follow the cryptodiagnosis approach of Bean’s work on Kryptos K4: measure what the ciphertext is, enumerate the cipher classes it could be, and eliminate each

one against a calibrated null. The value of a diagnosis is a smaller, sharper hypothesis space and an honest account of what is and is not ruled out.

The paper makes three contributions. First, a precise statistical characterisation of the unsolved stream and a generative model for its one anomaly. Second, a control-validation methodology — a *detection floor* and a *matched ceiling* — that guards an elimination against both false negatives and false positives, with each failure mode caught in our own early work. Third, a broad, reproducible elimination ledger, cross-validated against two independent projects.

## 2 The corpus and its provenance

We work from the community transcription vendored at `scream314/cicada3301`, parsed into 25 segments of about 15,750 runes. The unsolved LP2 material is 12,956 runes across 13 bundled segments. A second, independent transcription (`rtkd/iddqd`) agrees with ours on all but about eleven runes book-wide (99.95%), and reproduces every one of the 86 residual doublets discussed below (§23). The two likely share lineage, so this establishes stability rather than full independence.

Gematria Primus assigns each of 29 runes a letter or digraph and an ascending prime (2, 3, 5, ..., 109). The alphabet is prime-length, which matters: every non-zero shift and every multiplier is invertible modulo 29.

## 3 The verified cipher conventions

The solved pages fix the conventions, and we reproduce every one of them by forward encryption. Our `validate_solved.py` encrypts the known plaintext of each solved page under its stated scheme and matches the published ciphertext rune-for-rune (9 of 9 checks). Forward encryption is necessary because one convention makes decryption ambiguous.

The conventions are: encryption is  $c = (p + k) \bmod 29$  in rune-index space; atbash is the reversed alphabet,  $i \mapsto 28 - i$ ; Vigenère uses keys taken from riddle answers; one page uses a totient stream  $\varphi(\text{prime})$ . The **literal- $\mathcal{P}$  rule** is the ambiguous one: a plaintext  $F$  is written as an unencrypted  $\mathcal{P}$  and consumes no key, but ordinary encryption can also yield  $\mathcal{P}$  (for example  $M + I \equiv 0$ ), so a ciphertext  $\mathcal{P}$  has two readings. This is why validation encrypts forward.

## 4 The central finding: a uniform stream with one constraint

The unsolved stream has an index of coincidence of 1.000 — indistinguishable from random. Its bigrams are flat. The single departure from randomness is at lag 1: the same rune almost never occurs twice in a row.

We measured the doublet rate at 0.66% (86 equal-adjacent pairs in 12,934), against a random expectation of  $1/29 = 3.45\%$  (§30). The indicators are pairwise independent, so the variance is exact; the deficiency is  $z \approx -17.4$ . Differences at lags 2 through 8 are null. The non-zero first differences are uniform to  $p \approx 0.04$ . So the anomaly is a *pure lag-1 effect* and nothing else.

This rules out, on its own, substitution and every periodic or independent keystream, because those preserve either the unigram or the bigram structure that the stream lacks. What survives is a mechanism that acts at the cipher’s *output*: a no-repeat enforcement.

Two output mechanisms produce a flat IoC with suppressed doublets. A **re-roll** pad picks a different key value when the output would repeat; the keystream stays locked to position. A **key-skip** advances the key pointer an extra step into a fixed keystream to dodge the repeat; the keystream *desynchronises*. Both match the statistics, but they differ for an attacker. Re-roll leaves a known keystream testable position-by-position. Key-skip consumes an extra, invisible key value at every avoided doublet, which desynchronises the stream by about 3% and defeats crib-dragging and every periodicity test — exactly what we observe.

#### 4.1 The soft-rejection refinement

A hard no-repeat rule would leave zero doublets. The stream has 86. We show these are not transcription noise but signal. §23 established that a second transcription reproduces all 86, so copy error at the required rate is not supported. §44 gives the mechanism: the rule is *soft*. It keeps a would-be doublet with a fixed acceptance probability. Fitting that probability on our stream,

$$p_{\text{keep}} = 86 \text{ observed} / (12,934 \text{ pairs} / 29 \text{ expected collisions}) = 0.193,$$

and a simulation at  $p = 0.19$  reproduces both the flat IoC and the 0.66% rate. A parallel project (below) fitted the same figure ( $\approx 0.18$ ) independently. The 86 doublets are the filter’s acceptance leak. Combinatorially the construction is a Smirnov word / Carlitz composition — a sequence with a bias against equal neighbours. This is orthogonal to the resolution distribution, which §11 found uniform (a deterministic bump or nearest-value nudge is ruled out).

### 5 Methodology: control-validated elimination

An elimination claim has the form “if cipher X were the answer, we would have seen it.” That claim is testable, and if it is not tested the negative is worthless. A self-audit of our own early work (§28–§31) found both failure modes, so we state the methodology as the paper’s second contribution.

**The detection floor.** For each hypothesis, plant it — encrypt known English under that scheme — and recover it through the same pipeline. The minimum score over hypotheses that recover is the floor: the score a genuine break would produce. A hypothesis that fails to recover when planted is reported as *not covered* (no evidence), never as a negative. This guards against **false negatives**: an arbitrary “near-English” threshold once rejected a script’s own successful control, because a beam decoder’s per-skip penalty puts a real break well below plain English.

**The matched ceiling.** Draw a null of the same length, over the same number of trials, scored by the same function. This guards against a ceiling biased low by too few trials or by a length mismatch — shorter sequences score higher.

**The coverage and margin gates.** If few hypotheses are identifiable, the floor rests on a handful of samples and the search is demonstrably fitting wrong hypotheses to planted data; below half coverage the honest verdict is *no evidence*. If the floor nearly touches the ceiling, no score separates a break from noise. Both gates guard against **false positives**: two “possible breaks” in our work were gibberish decodes that out-scored real English of the same length, and both dissolved under these gates.

The null must also be independent of the search. One attack drew its ceiling from the very generator it brute-forced, inflating the ceiling by 1.36 nats and hiding about a third of genuine breaks

(§30). Our shared `controls.py` draws nulls from a domain-separated hash stream that no searched generator can produce.

## 6 The elimination ledger

Each entry below passed a positive control and, where relevant, a matched ceiling. Scores are per-symbol trigram log-likelihoods unless noted; a genuine break lands near the detection floor, gibberish near the ceiling.

**Substitution and periodic keys (§3).** Monoalphabetic substitution, shifts and atbash are excluded by  $\text{IoC} = 1.000$ . Repeating-key Vigenère is excluded for every period up to 40 by the periodic-IoC scan and a key-skip beam.

**Prime and totient keystreams (§3).** Both signs, with and without the key-skip, negative.

**Autokey (§3).** Plaintext- and ciphertext-fed, short primers, with a planted control added by audit.

**Running keys (§6, §9, §36, §37).** We tested the texts Cicada is documented to have used or referenced — the King James Bible, Crowley’s *Liber AL vel Legis*, the *Mabinogion*, Blake’s *Marriage of Heaven and Hell*, Emerson’s essays, and the Anglo-Saxon Rune Poem — whole text, every offset, both signs, and read backwards as well as forwards. Every control recovers a key planted in that text at about 100%; every real decode falls short. We also measured a power law: the larger the key text, the more offsets the scan can cherry-pick, so pure chance climbs toward the score a real key would give (0.71 at 2.4k runes for the Rune Poem, 0.45 at 407k for Emerson, 0.36 at 3.0M for the KJV). Texts of tens of millions of runes are progressively untestable, which is a general limit on the running-key approach, not a property of any one text.

**Word and coined keys (§7, §8, §39).** The Cicada vocabulary plus the top ~1,200 English words, and single- and double-transform coined variants (the FIRFUMFERENFE = CIRCUMFERENCE family), all with the key-skip, negative. At these lengths the search overfits any text, which is the limit of the whole word-key family.

**Non-additive functions (§48, §54).** The clean invertible non-additive family on a prime alphabet is the affine cipher  $c = a \cdot p + k$ . Every multiplier  $a$  in  $1 \dots 28$ , over the prime, totient and word keystreams, negative; the control recovers the planted multiplier. Bitwise XOR is undefined on 29 symbols and a pure multiplicative cipher fails on  $\mathcal{P} = 0$ , so affine subsumes the family. mortlach’s gematria-rotation space collapses onto this: a rotation is absorbed into the additive key, atbash is  $a = 28$ , and “L2R/R2L transposition” is a reading direction. Every orientation of the ciphertext — forward, reversed, atbash, atbash-reversed — is negative. Arbitrary rune transposition is *un-searchable* here: it preserves unigram frequencies and our bigrams are flat, so there is no statistical handle, the opposite of the Zodiac Z340 case whose period-19 bigram spike revealed its transposition.

**Difference-space cumulative ciphers (§12, §38).** If each rune were added to the one before it, the differences would be the message. We tested keyless, repeating-key and prime/totient recovery on  $d[i] = c[i] - c[i-1]$ , and word and book keys on  $d$ ; the difference stream is as random as the stream itself.

**Seeded and derived pads (§13, §41).** Seven pseudo-random generators (glibc, NR, Java LCGs, xorshift, Mersenne, hash-of-counter) over small and thematic seeds, both signs, under the re-roll

model, negative against a chance ceiling. The *derived-seed* lane — a keystream grown from a short seed by a hash in counter mode — is the sharp case. A rigid decoder scores a correct seed as noise, but a skip-aware beam recovers a planted CICADA3301 SHA-256-CTR seed at 100%. So the lane is inside resolving power. Over 116 thematic passphrase seeds and four hash constructions the real decode is negative (floor  $-3.73$ , ceiling  $-4.18$ , best real  $-4.26$ ). Low-entropy thematic seeds are ruled out; a high-entropy seed is not.

**Hand-computable keystreams (§42, §50).** The Gromark cipher grows a long key from a short primer by chain addition. Length-2 primers are the Fibonacci recurrence mod 29 (period  $\leq 14$ ), already covered by the periodic scan; length-3 (period 871) is negative. Integer sequences (Fibonacci, Lucas, Catalan, factorial, partition, and arithmetic progressions) as keystreams are negative; and a no-output-rule doublet-suppressing keystream is refuted in principle, because a constant-difference key leaves the English difference structure in the ciphertext first differences, which are in fact flat.

**Generalised interrupter (§40, §55, §56).** Every prior interrupter test assumed the pause-rune is  $\mathcal{P}$ . A power analysis shows the generalisation to all 29 runes is detectable in principle (an oracle gives per-slot IoC 1.886 against a  $\sim 1.0$  floor) but that 63% of ciphertext runes equal to a candidate interrupt rune are coincidental, and each false skip desynchronises the rest. *relikd's LiberPrayground* ran the full interrupt-subset search over all 29 runes and key lengths 1–32 and ships it as a database. We verified it firsthand: it recovers every solved control page near English (db\_norm 0.81–1.00), and the one solved LP2 page that sits in the unsolved numbering scores 0.997, while every genuinely-unsolved section tops out at 0.55–0.63. The alternating-alphabet ("modulo") variant is likewise negative; its higher-looking per-subgroup scores are short-slot length inflation.

**Structural and positional tests (§34, §45, §46, §49).** Per-page and per-line key resets are a decisive negative — the winning schedule on real text is the one that never resets. Word-, line- and page-initial rune distributions are uniform, so there is no word/page-synchronised key and no acrostic; a line-initial anomaly failed against an independent, image-derived line segmentation and is a transcription artifact. Every exact ciphertext repeat, mined book-wide and weighed against a matched Smirnov null, is coincidental, so there are no Kasiski anchors.

**Magic squares, code pages and the stream as a pad (§16, §17, §22, §24, §47).** The page-16 and page-32 magic squares are not a key, an interrupter schedule, or a standalone message. The code pages (66–68) are a 256-byte object, not a substituted message (see §8 below); tried as a pad, an index, a self-cipher and a lookup table, all negative. The unsolved stream tested as key material rather than ciphertext — a running key against candidate texts and its own reflections — is closed by uniformity: a uniform stream minus any independent text stays uniform, and a folded pad would force a palindrome the stream is not.

**Pre-2014 numeric hints (§26, §51).** The unused "hints never used" numerics — whitespace prime sequences, the onion cookies 167 and 761, and the 128-digit 2012 postscript number — as keys, offsets, hash seeds and autokey primers, negative.

## 7 Cross-validation against independent projects

Two other rigorous efforts reach the same central finding, which strengthens the diagnosis beyond one toolkit. The community's own frequency analysis reports the identical doublet table and the "only-anomaly" verdict. *relikd's InterruptDB* is a large independent negative for the interrupter family (§6). *Dukotah's cicada3301* project, which pins the same ciphertext by SHA-256, agrees

to within nine runes (the transcription-lineage delta), reaches the soft-rejection figure  $p \approx 0.18$  independently, and its ledger agrees with ours on every shared negative (§44). One methodological point favours our results: where that project voided its own key-text negatives as unsound (its decoder was validated on the wrong mechanism), ours use the skip-aware beam with a passing control and stand.

## 8 The numeric content and the code pages

The transcription omits three non-runic pages (66–68) that carry 256 two-character codes. An independent archive renders the same pages as a 256-byte hexadecimal string ("String 4"), of the same class as byte strings that elsewhere encode Tor hidden-service addresses. We derived the code-to-byte map —  $\text{byte} = \text{digit} \cdot 60 + \text{base62}(\text{char})$  — and our codes reproduce that 256-byte string exactly, after correcting three lowercase-l / capital-I transcription errors, one of which the parallel project had independently flagged (§53). This verifies the transcription byte-for-byte against an independent rendering. It does not decode the bytes: they are high-entropy and behave as key material behind the same pad, not a message.

## 9 The information-theoretic wall

Shannon's unicity distance is  $U = H(K) / D$ , where  $D$  is the plaintext redundancy per symbol. For digraph-transliterated English,  $D \approx 3$  bits per rune. So a 128-bit seed determines the plaintext after about 43 runes, a 256-bit seed after about 85. At 12,956 runes the ciphertext is hundreds of times past unicity for any short-seed scheme: the plaintext is uniquely determined in principle, and any resistance is computational, never information-theoretic, unless the key entropy approaches  $N \cdot \log_2 29 \approx 63$  kbits — a true pad.

This is the correct formulation of the wall. The stream is "OTP-class": its ciphertext cannot distinguish a true external pad from a keystream derived from a short seed, and only the derived-seed case is attackable, because its key space is finite. Our derived-seed search rules out low-entropy thematic seeds (§6). What remains is seed entropy, not the pad idea. A high-entropy or external pad is the wall the runes sit behind.

## 10 Conclusion

The unsolved *Liber Primus* is a uniform stream bound by a single soft no-repeat rule with acceptance rate  $p \approx 0.19$ . Every classical and modern cipher class we could formulate is eliminated against a planted control and a matched ceiling, and two independent projects reach the same negative. The plaintext is far past unicity, so the wall is computational: a high-entropy or external keyed pad.

Three directions could move the diagnosis. First, external key material — a signed pointer, an archived text, or the numeric content — supplied from outside the runic stream; the code pages behave as key material behind their own pad and are the way around, not through, the wall. Second, a longer or independent line and page segmentation, which would let a depth or key-reuse test run that the current unit lengths cannot support (§43). Third, the community: a published cryptodiagnosis recruits the readers who might hold the external material the cipher appears to require.

The contribution is the narrowed hypothesis space and the control-validation method, not a break.

A clean negative with a control is the honest unit of progress on a cipher that a decade has not broken.

## 11 Data and code availability

All data, scripts, and per-section results are at <https://github.com/jens-wedin/liber-primus>. The statistical claims are reproduced by `validate_solved.py` (the solved-page checks), `no_repeat_model.py` (the mechanism and the `p_keep` fit), `analyze_interruptdb.py` (the `relikd` verification), and the `attack_*` scripts (the elimination ledger), each routed through `controls.py`. A `pytest` suite pins the invariants and the positive controls.

## 12 References

Peer-reviewed sources:

1. C. E. Shannon. Communication Theory of Secrecy Systems. *Bell System Technical Journal*, 28(4):656–715, 1949.
2. R. Bean. Cryptodiagnosis of "Kryptos K4". In *Proceedings of the 4th International Conference on Historical Cryptology (HistoCrypt 2021)*, Linköping Electronic Conference Proceedings, vol. 183, pp. 25–33. Linköping University Electronic Press, 2021. DOI: 10.3384/ecp183153.
3. D. Oranchak, S. Blake, J. Van Eycke. The Solution of the Zodiac Killer’s 340-Character Cipher (Z340). arXiv:2403.17350, 2024.
4. G. Lasry. *A Methodology for the Cryptanalysis of Classical Ciphers with Search Metaheuristics*. Kassel University Press, 2018.

Software and data:

1. R. Speer. `rspeer/wordfreq` (v3.0). Zenodo, 2022. DOI: 10.5281/zenodo.7199437.
2. J. Wedin. *Liber Primus cryptanalysis toolkit*. <https://github.com/jens-wedin/liber-primus>, 2026. All statistics, the elimination ledger, and the tests in this paper are reproduced here.

Community and independent-project sources:

1. Transcriptions and archives: `scream314/cicada3301` (the primary rune transcription); `rtkd/iddqd` (the cross-check transcription and the "byte strings" rendering of the code pages); `krisyotam/cicada3301` (provenance-tiered image archive).
2. Independent solver projects reaching the same central finding: `relikd/LiberPrayground` (the InterruptDB, the interrupter and modulo sweeps); `Dukotah/cicada3301` (a parallel control-validated project; the soft-rejection fit and the elimination ledger cross-check).
3. Reference material: the Uncovering Cicada wiki (<https://uncovering-cicada.fandom.com>); the CicadaSolvers quickstart briefing (<https://www.cicadasolvers.com/quickstart/>); the *Cicada 3301* entry, *Wikipedia*.

## 13 Acknowledgements

This diagnosis stands on a decade of open community work. I thank the maintainers of the transcriptions this paper depends on — `scream314` (the primary rune transcription) and `rtkd` (the independent cross-check and the byte-string rendering of the code pages) — and the CicadaSolvers and Uncovering Cicada communities for the frequency analysis and the puzzle history. Two independent solver projects were essential to the cross-validation in §7: `relikd`’s *LiberPrayground*, whose finished InterruptDB let me verify the interrupter and modulo families without a 38-hour rebuild, and `Dukotah`’s *cicada3301*, whose control-validated ledger and soft-rejection fit corroborate the central finding on the same ciphertext object. Any errors are my own.

## 14 Appendix A. Gematria Primus

The 29-rune alphabet, in its canonical order. Each rune maps to one English letter or digraph and to the  $n$ th prime in ascending order. Index  $i$  is the value used for the modular arithmetic in the body ( $c = p + k \bmod 29$ ).

| i  | rune | letter | prime | i  | rune | letter | prime |
|----|------|--------|-------|----|------|--------|-------|
| 0  | 𐀀    | F      | 2     | 15 | 𐀑    | S      | 53    |
| 1  | 𐀁    | U      | 3     | 16 | 𐀒    | T      | 59    |
| 2  | 𐀂    | TH     | 5     | 17 | 𐀓    | B      | 61    |
| 3  | 𐀃    | O      | 7     | 18 | 𐀔    | E      | 67    |
| 4  | 𐀄    | R      | 11    | 19 | 𐀕    | M      | 71    |
| 5  | 𐀅    | C      | 13    | 20 | 𐀖    | L      | 73    |
| 6  | 𐀆    | G      | 17    | 21 | 𐀗    | NG     | 79    |
| 7  | 𐀇    | W      | 19    | 22 | 𐀘    | OE     | 83    |
| 8  | 𐀈    | H      | 23    | 23 | 𐀙    | D      | 89    |
| 9  | 𐀉    | N      | 29    | 24 | 𐀚    | A      | 97    |
| 10 | 𐀊    | I      | 31    | 25 | 𐀛    | AE     | 101   |
| 11 | 𐀋    | J      | 37    | 26 | 𐀜    | Y      | 103   |
| 12 | 𐀌    | EO     | 41    | 27 | 𐀝    | IA     | 107   |
| 13 | 𐀍    | P      | 43    | 28 | 𐀞    | EA     | 109   |
| 14 | 𐀎    | X      | 47    |    |      |        |       |

Transliteration is per word: the digraphs (TH, EO, NG, OE, AE, IA, EA) are recognised inside a word but never across a word boundary, and the reductions  $K \rightarrow C$ ,  $V \rightarrow U$ ,  $Q \rightarrow CW$  apply. Atbash is the order-reversal  $i \mapsto 28 - i$ . The prime column is the basis of the “Gematria sum” numerology on the solved pages; it plays no role in the elimination results of this paper.